

Computer Architecture

Introduction and Basics

So, I Hope You Are Here for This



"C" as a model of computation

Programmer's view of how
a computer system works

- How does an assembly program end up executing as digital logic?
- **What happens in-between?**
- How is a computer designed using logic gates and wires to satisfy specific goals?

*Architect/microarchitect's view:
How to design a computer that
meets system design goals.*

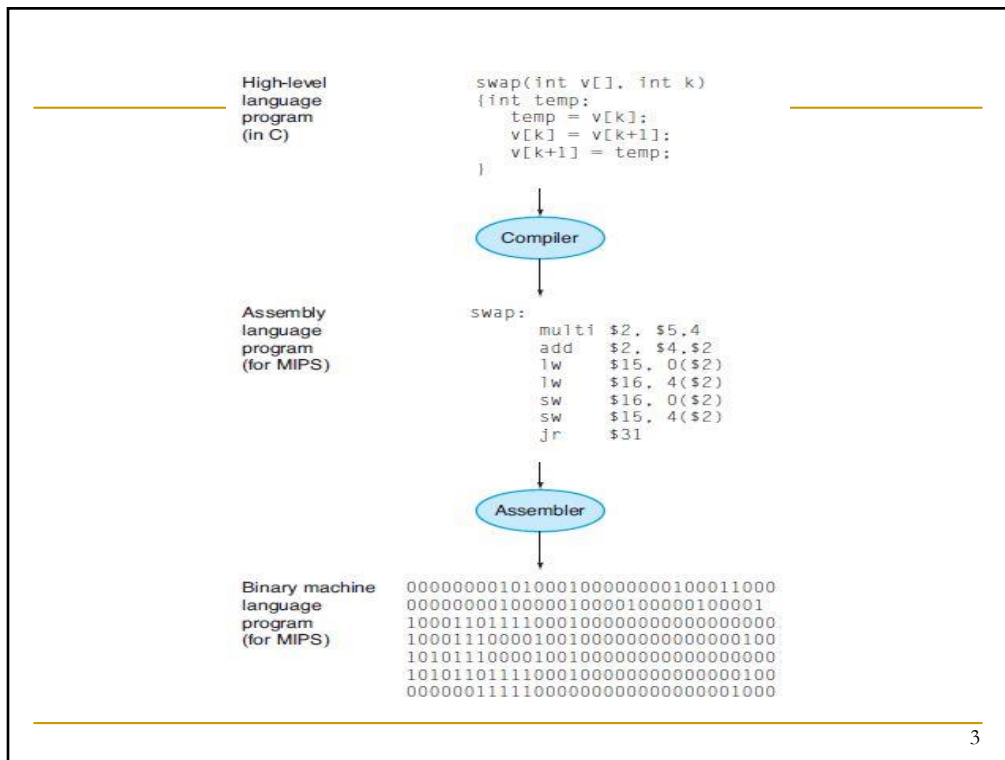
*Choices critically affect both
the SW programmer and
the HW designer*



HW designer's view of how
a computer system works

Digital logic as a
model of computation

How is the HW/SW interface designed



Levels of Transformation

"The purpose of computing is insight" (Richard Hamming)
, not numbers

We gain and generate insight by solving problems
*How do we **ensure** problems are solved by **electrons**?*



Problem
Algorithm
Program/Language
Runtime System (VM, OS, MM)
ISA (Architecture)
Microarchitecture
Logic
Circuits
Electrons

4

ISA is the interface between hardware and software... It is a contract that the hardware promises to satisfy.

Algorithm: step by step procedure where each step is effectively computable (by a computer), is definite (precisely defined) – “do until fast” is not definite, and terminates

Hamming distance: number of locations in which the corresponding symbols of two equal-length strings is different

[Hamming, Richard W.](#) (1950), "[Error detecting and error correcting codes](#)", [Bell System Technical Journal](#) **29** (2): 147–160

Hamming codes

The Power of Abstraction

- Levels of transformation create abstractions

- **Abstraction:** A higher level only needs to know about the interface to the lower level, not how the lower level is implemented
- E.g., high-level language programmer does not really need to know what the ISA is and how a computer executes instructions

- Abstraction improves productivity

- No need to worry about decisions made in underlying levels
- E.g., programming in Java vs. C vs. assembly vs. binary vs. by specifying control signals of each transistor every cycle

- Then, why would you want to know what goes on underneath or above?

Crossing the Abstraction Layers

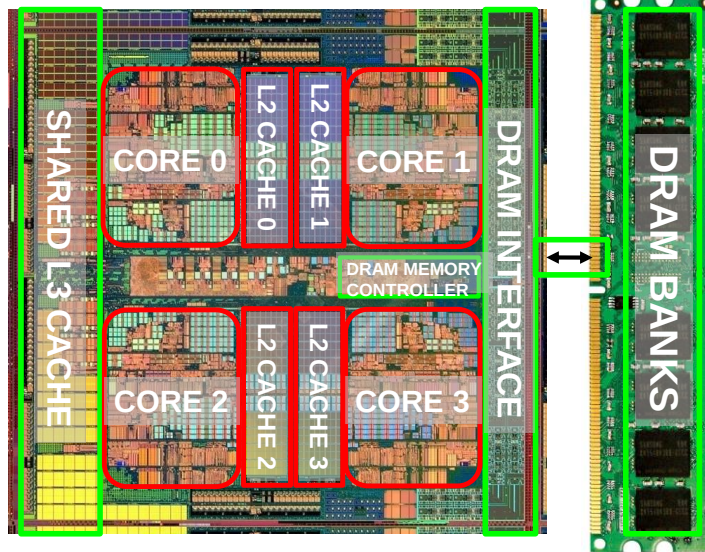
- As long as everything goes well, not knowing what happens in the underlying level (or above) is not a problem.
 - What if
 - The program you wrote is running slow?
 - The program you wrote does not run correctly?
 - The program you wrote consumes too much energy?
 - What if
 - The hardware you designed is too hard to program?
 - The hardware you designed is too slow because it does not provide the right primitives to the software?
 - What if
 - You want to design a much more efficient and higher performance system?
-

Crossing the Abstraction Layers

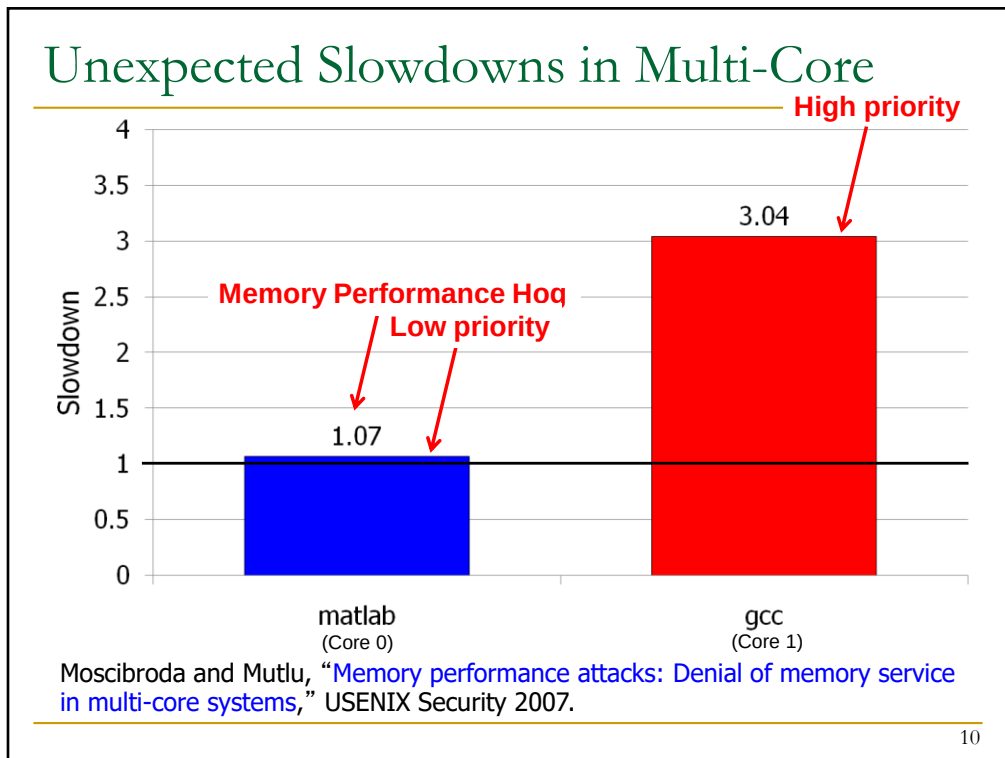
- Two key goals of this course are
 - to understand how a processor works underneath the software layer and how decisions made in hardware affect the software/programmer
 - to enable you to be comfortable in making design and optimization decisions that cross the boundaries of different layers and system components

An Example: Multi-Core Systems

Multi-Core
Chip



*Die photo credit: AMD Barcelona



What kind of performance do we expect when we run two applications on a multi-core system? To answer this question, we performed an experiment. We took two applications we cared about, ran them together on different cores in a dual-core system, and measured their slowdown compared to when each is run alone on the same system. This graph shows the slowdown each app experienced. (DATA explanation...)

Why do we get such a large disparity in the slowdowns?

Is it the priorities? No. We went back and gave high priority to gcc and low priority to matlab. The slowdowns did not change at all. Neither the software or the hardware enforced the priorities.

Is it the contention in the disk? We checked for this possibility, but found that these applications did not have any disk accesses in the steady state. They both fit in the physical memory and therefore did not interfere in the disk.

What is it then? Why do we get such large disparity in slowdowns in a dual core system?

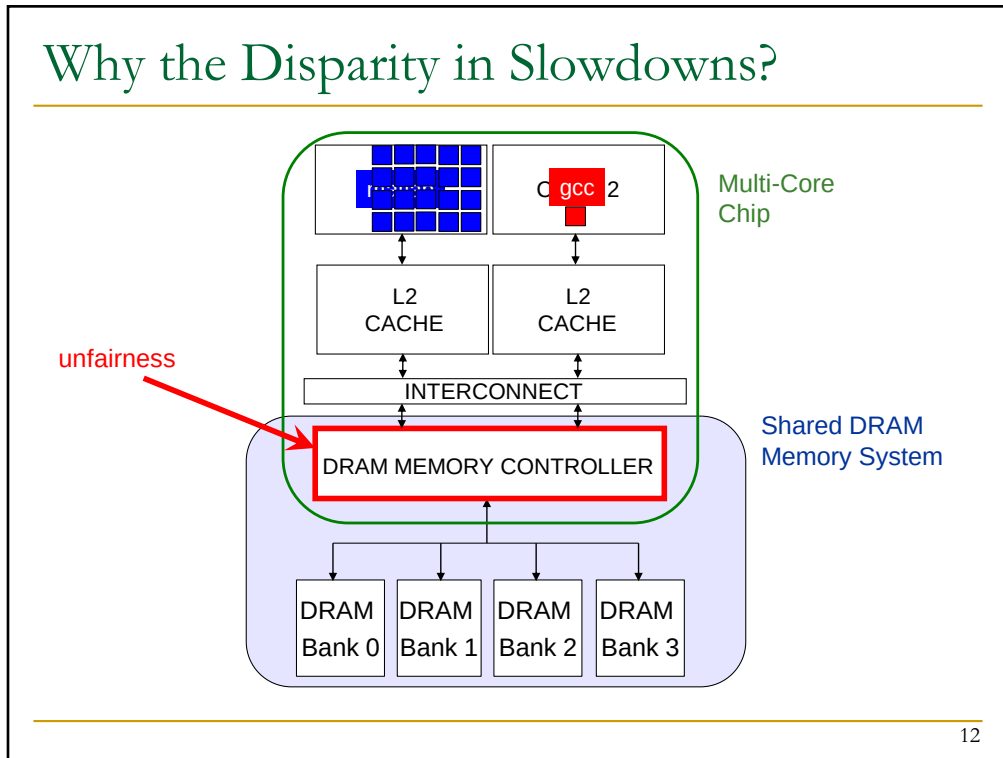
I will call such an application a "memory performance hog"
Now, let me tell you why this disparity in slowdowns happens.

Is it that there are other applications or the OS interfering with gcc, stealing its time quantum? No.

A Question or Two

- Can you figure out why there is a disparity in slowdowns if you do not know how the system executes the programs? **No**
- Can you fix the problem without knowing what is happening “underneath”? **No**

Why the Disparity in Slowdowns?



12

-In a multi-core chip, different cores share some hardware resources. In particular, they share the **DRAM memory system**. The shared memory system consists of this and that.

-When we run matlab on one core, and gcc on another core, both cores generate memory requests to access the DRAM banks. When these requests arrive at the DRAM controller, the controller favors matlab's requests over gcc's requests. As a result, matlab can make progress and continues generating memory requests. These requests are again favored by the DRAM controller over gcc's requests. Therefore, gcc starves waiting for its requests to be serviced in DRAM whereas matlab makes very quick progress as if it were running alone.

-Why does this happen? This is because the algorithms employed by the DRAM controller are unfair.

-But, why are these algorithms unfair? Why do they unfairly prioritize matlab accesses?

-To understand this, we need to understand how a DRAM bank operates.

-Almost all systems today contain multi-core chips

-Multi-core systems consist of multiple on-chip cores and caches

-Cores share the DRAM memory system

-DRAM memory system consists of

-DRAM banks that store data (multiple banks to allow parallel accesses)

-DRAM memory controller that mediates between cores and DRAM memory

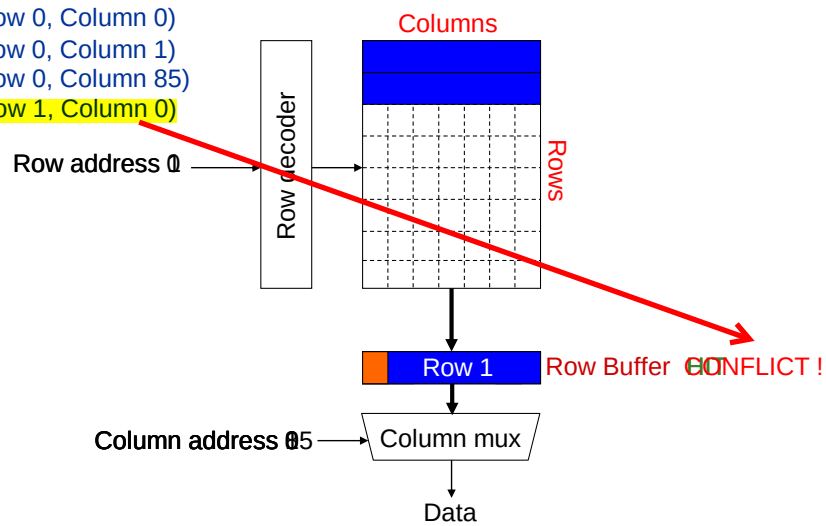
-It schedules memory operations generated by cores to DRAM

-This talk is about exploiting the unfair algorithms in the memory controllers to perform denial of service to running threads

-To understand how this happens, we need to know about how each DRAM bank operates

DRAM Bank Operation

Access Address:
(Row 0, Column 0)
(Row 0, Column 1)
(Row 0, Column 85)
(Row 1, Column 0)



DRAM Controllers

- A **row-conflict** memory access takes significantly longer than a row-hit access
- Current controllers take advantage of the row buffer
- Commonly used scheduling policy (FR-FCFS) [Rixner 2000]*
First-Ready, First-Come-First-Service
 - (1) Row-hit first: Service row-hit memory accesses first
 - (2) Oldest-first: Then service older accesses first
- This scheduling policy aims to maximize DRAM throughput

*Rixner et al., "Memory Access Scheduling," ISCA 2000.

*Zuravleff and Robinson, "Controller for a synchronous DRAM ...," US Patent 5,630,096, May 1997.

The Problem

- Multiple applications share the **DRAM controller**
- DRAM controllers designed to maximize DRAM data throughput
- DRAM scheduling policies are unfair to some applications
 - Row-hit first: unfairly prioritizes apps with **high row buffer** locality
 - Threads that keep on accessing the same row
 - Oldest-first: unfairly prioritizes **memory-intensive applications**
- DRAM controller vulnerable to denial of service attacks
 - Can write programs to exploit unfairness

A Memory Performance Hog

```
// initialize large arrays A, B
for (j=0; j<N; j++) {
    index = j*linesize; streaming
    A[index] = B[index];
    ...
}
```

STREAM

- Sequential memory access
- Very high row buffer locality (96% hit rate)
- Memory intensive

```
// initialize large arrays A, B
for (j=0; j<N; j++) {
    index = rand(); random
    A[index] = B[index];
    ...
}
```

RANDOM

- Random memory access
- Very low row buffer locality (3% hit rate)
- Similarly memory intensive

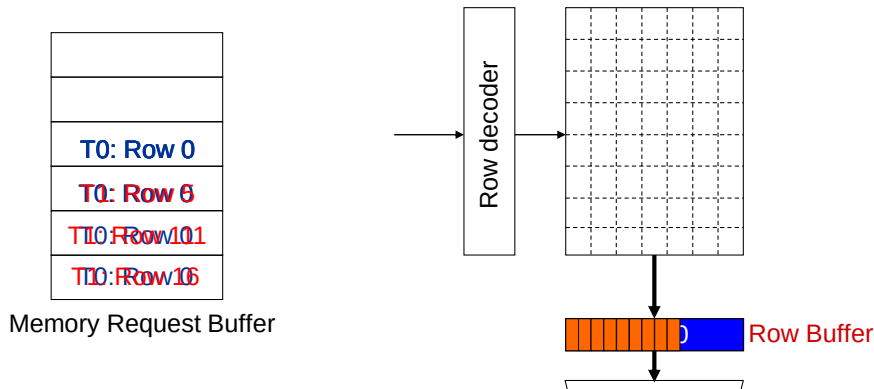
Moscibroda and Mutlu, “[Memory Performance Attacks](#),” USENIX Security 2007.

16

- Streaming through memory by performing operations on two 1D arrays.
- Sequential memory access
- Each access is a cache miss (elements of array larger than a cache line size) → hence, very memory intensive

Link to the real code...

What Does the Memory Hog Do?



Row size: 8KB, cache block size: 64B
 128 (8KB/64B) requests of T0 serviced before T1

Moscibroda and Mutlu, "Memory Performance Attacks," USENIX Security 2007.

17

- Pictorially demonstrate how stream denies memory service to rdarray
- Stream continuously accesses columns in row 0 in a streaming manner (streams through a row after opening it)
- In other words almost all its requests are row-hits
- RDarray's requests are row-conflicts (no locality)
- The DRAM controller reorders streams requests to the open row over other requests (even older ones) to maximize DRAM throughput
- Hence, rdarray's requests do not get serviced as long as stream is issuing requests at a fast-enough rate
- In this example, the red thread's request to another row will not get serviced until stream stops issuing a request to row 0
- With those parameters, 128 requests of stream would be serviced before 1 from rdarray
- As row-buffer size increases, which is the industry trend, this problem will become more severe
- This is not the worst case, but it is easy to construct and understand
 - Stream falls off the row buffer at some point
- I leave it to the listeners to construct a case worse than this (it is possible)

Now That We Know What Happens Underneath

- How would you solve the problem?
- What is the right place to solve the problem?
 - ❑ Programmer?
 - ❑ System software?
 - ❑ Compiler?
 - ❑ Hardware (Memory controller)?
 - ❑ Hardware (DRAM)?
 - ❑ Circuits?

Problem
Algorithm
Program/Language
Runtime System (VM, OS, MM)
ISA (Architecture)
Microarchitecture
Logic
Circuits
Electrons

- Two other goals of this course:
 - ❑ Enable you to **think critically**
 - ❑ Enable you to **think broadly**

Takeaway

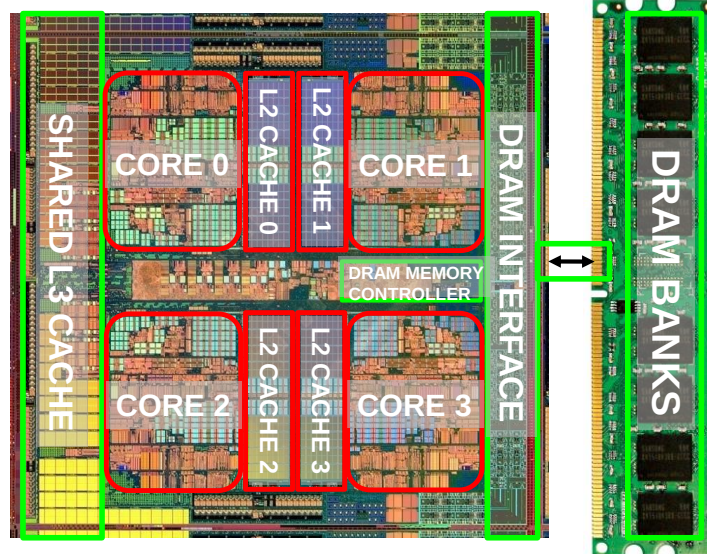
- Breaking the abstraction layers (between components and transformation hierarchy levels) and knowing what is underneath enables you to solve problems

Another Example

- DRAM Refresh

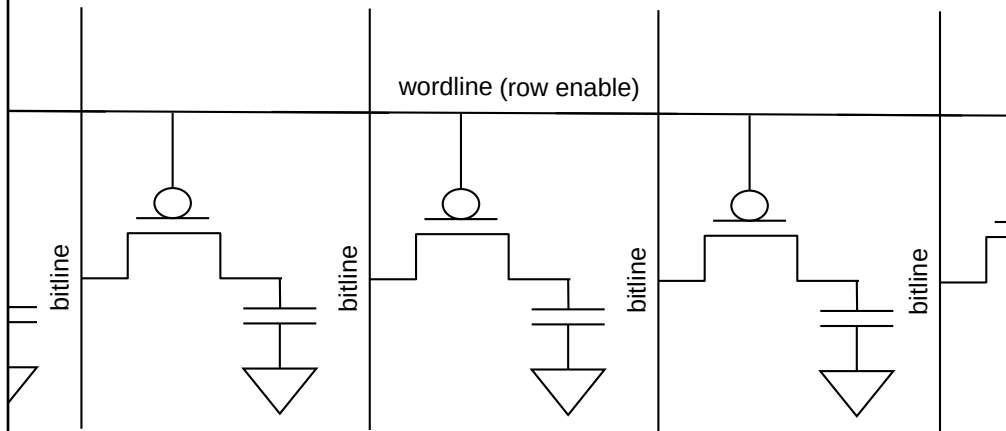
DRAM in the System

Multi-Core
Chip



*Die photo credit: AMD Barcelona

A DRAM Cell



- A DRAM cell consists of a **capacitor** and an **access transistor**
- It stores data in terms of charge in the capacitor
- A DRAM chip consists of (10s of 1000s of) rows of such cells

SAFARI

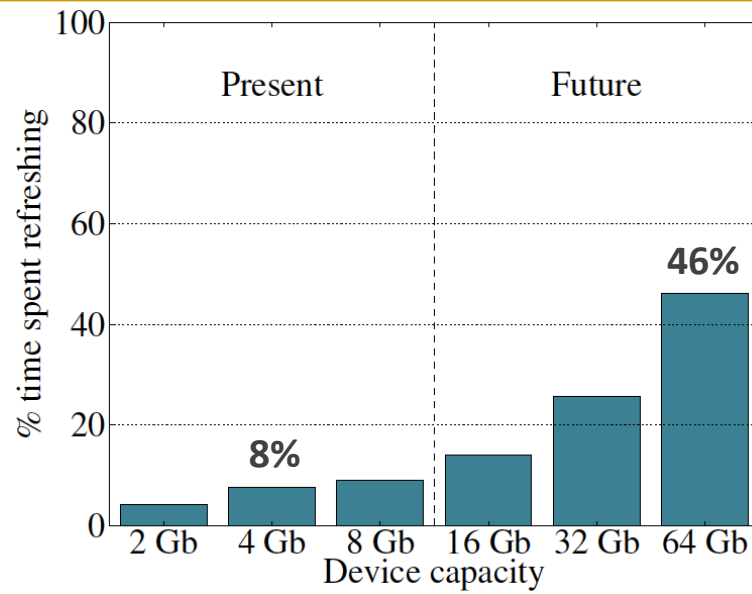
DRAM Refresh

- DRAM capacitor charge leaks over time
- The memory controller needs to refresh each row periodically to restore charge
 - Activate each row every N ms
 - Typical N = 64 ms
- Downsides of refresh
 - **Energy consumption**: Each refresh consumes energy
 - **Performance degradation**: DRAM rank/bank unavailable while refreshed
 - **QoS/predictability impact**: (Long) pause times during refresh
 - **Refresh rate limits DRAM capacity scaling**

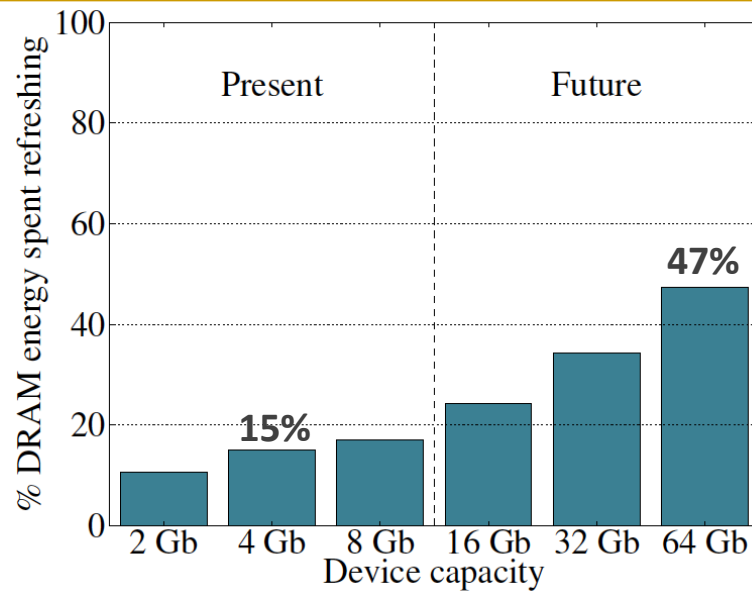
First, Some Analysis

- Imagine a system with 1 ExaByte DRAM
- Assume a row size of 8 KiloBytes
- How many rows are there?
- How many refreshes happen in 64ms?
- What is the total power consumption of DRAM refresh?
- What is the total energy consumption of DRAM refresh during a day?
- Part of your Homework 1

Refresh Overhead: Performance



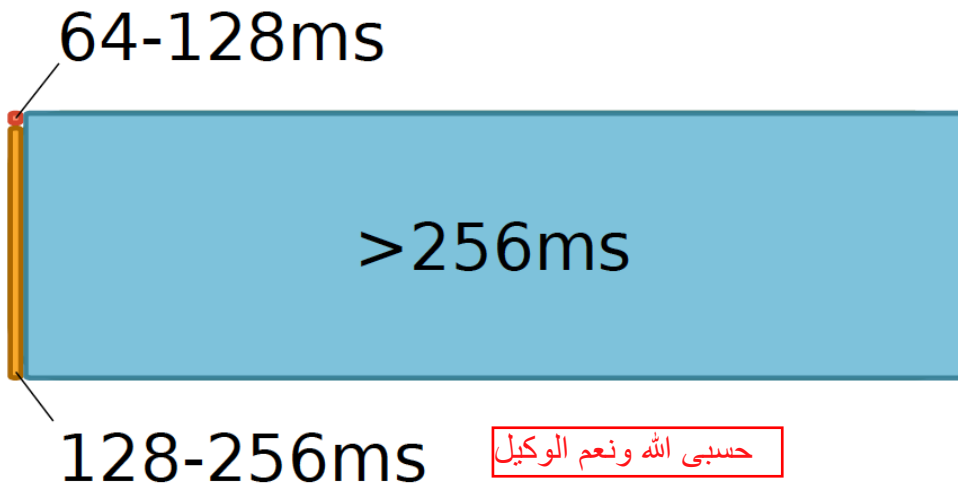
Refresh Overhead: Energy



How Do We Solve the Problem?

- Do we need to refresh all rows every 64ms? حسبى الله ونعم الوكيل
- What if we knew what happened underneath and exposed that information to upper layers?

Underneath: Retention Time Profile of DRAM



Taking Advantage of This Profile

- Expose this retention time profile information to
 - the memory controller حسبى الله ونعم الوكيل
 - the operating system
 - the programmer?
 - the compiler?
- How much information to expose?
 - Affects hardware/software overhead, power consumption, verification complexity, cost
- How to determine this profile information?
 - Also, who determines it?

An Example: RAIDR

- Observation: Most DRAM rows can be refreshed much less often without losing data [Kim+, EDL'09][Liu+ ISCA'13]

- Key idea: Refresh rows containing weak cells more frequently, other rows less frequently

1. **Profiling:** Profile retention time of all rows

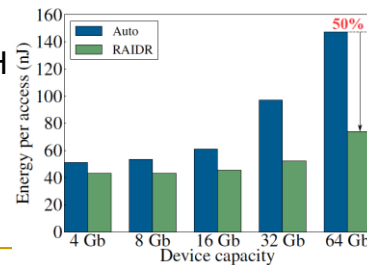
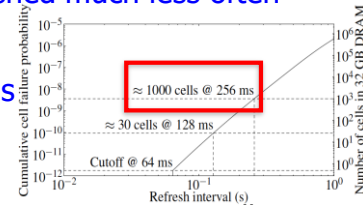
2. **Binning:** Store rows into bins by retention time in memory controller

Efficient storage with Bloom Filters (only 1.25KB for 32GB memory)

3. **Refreshing:** Memory controller refreshes rows in different bins at different rates

- Results: 8-core, 32GB, SPEC, TPC-C, TPC-H

- 74.6% refresh reduction @ 1.25KB storage
- ~16%/20% DRAM dynamic/idle power reduction
- ~9% performance improvement
- Benefits increase with DRAM capacity



Liu et al., "RAIDR: Retention-Aware Intelligent DRAM Refresh," ISCA 2012.

Takeaway

- **Breaking the abstraction layers** (between components and transformation hierarchy levels) and **knowing what is underneath** enables you to solve problems and design better future systems حسبى الله ونعم الوكيل
- Cooperation between multiple components and layers can enable more effective solutions and systems حسبى الله ونعم الوكيل